

Assignment 3: Optimization 2

CS 312 · Due date: Oct 20 by 1 PM (Quiz time) · Ungraded

Basic setting. Unless a problem states otherwise, experiments use the course language-modeling setup: train models for next-token prediction on the provided DCLM-style data, evaluate using validation loss, and compare runs under the same data, token budget, and evaluation protocol. Lower validation loss is better.

Instructions. Submit a short experimental report, with a *maximum of 20 pages*. For each problem, state the comparison you ran, the hyperparameters you swept, the best validation loss and diagnostics you found, and the interpretation you draw from the result. Tune the learning rate and other central hyperparameters enough that conclusions are not artifacts of a bad run. When a problem asks for a plot, include clear axes, labels, and enough context for the trend to be interpretable.

Code. Starter launchers for these problems live in `experiments/a3_optimization`. Each one lists the baseline runs for its problem; the TODOs mark what you need to build yourself.

Background

Optimization is the process of finding $\arg \min_{\theta} \ell(\theta)$ for some loss ℓ . Up until now, we have done very little to understand the structure and nature of $\ell(\theta)$, instead treating it as an inscrutable black box.

However, it will turn out that understanding ℓ well is quite important in building a better intuition for neural network training dynamics. Why do deep networks train reliably? Why do training curves look the way they do? Why do we have learning rate schedules at all? Answering these questions is difficult without a deeper understanding of the loss landscape, which we will attempt to build in this assignment.

The loss landscape $\ell(\theta)$ is highly complicated and stochastic. To focus our study, we are going to think about two main questions

- How many minima are there, and what do they look like?
- What is the shape of the loss on the way to a minimum?

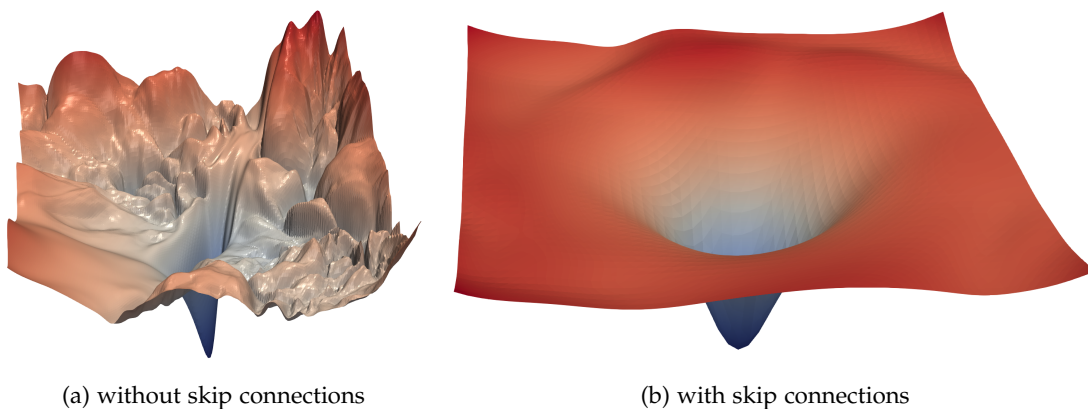
These will correspond to the study of minima and sharpness, respectively.

To begin with, we will take a more macroscopic view of this, and see how the loss reacts to simple interventions like averaging the model or changing the learning rate schedules. After we have built some intuition of how the losses behave, we will look at a much more local and fine-grained notion of loss structure by carefully inspecting the *Hessian* of the loss. This Hessian will reveal interesting and surprising facts about the loss landscape, and corroborate our macroscopic investigations.

Mode Connectivity

We start off with a study of *minima* in deep learning. The objective function in deep learning is nonconvex, and it is almost certain that there are many, many minima for any given deep learning objective.

In the early days of deep learning, this local minimum problem was one of the central problems in optimization. However, these issues seem much less salient today due to a combination of many factors, including the use of residual connections in deep networks, overparametrization, and high stochasticity in our optimizers.



Loss surfaces of ResNet-56 with and without skip connections. Figure 1 of [Li, Xu, Taylor, Studer, and Goldstein, “Visualizing the Loss Landscape of Neural Nets,” NeurIPS 2018.](#)

We will now study whether these interventions have eliminated local minima entirely, or if somehow “all minima are good minima” in a deep neural network. To see this, we will turn to an elementary fact in convex optimization.

For a convex function F , we have that,

$$F\left(\frac{x+y}{2}\right) \leq \frac{F(x) + F(y)}{2}$$

from Jensen's inequality. We can use this fact to build a simple probe for the minima structure of a deep neural network. We can run the stochastic optimization process multiple times, record our final iterates $(\theta_1, \theta_2, \dots)$, and study how the loss changes as a function of convex interpolations of these parameters ($\hat{\theta} = \sum_i \alpha_i \theta_i$ for α in the simplex).

Problem 1: Mode Connectivity

Two trained networks can have similar validation loss while living in very different parts of parameter space. In this problem, you will test when parameter-space averaging preserves validation loss.

Given two checkpoints θ_1 and θ_2 , define the midpoint barrier

$$B(\theta_1, \theta_2) = L\left(\frac{\theta_1 + \theta_2}{2}\right) - \frac{L(\theta_1) + L(\theta_2)}{2}.$$

Here L is validation loss. Use $B \leq 0.01$ as the criterion that weight averaging works for the two checkpoints.

Use the standard depth-8, 614M-token transformer setup with the default optimizer, but replace the default schedule with a constant learning rate after warmup.

The starter sets up this sweep with `TrainConfig` fields. One prefix run keeps its model and optimizer state at every x . Each branch forks from that state (`fork_from_run`, `fork_from_step`), uses its own data order (`batch_order_seed`), and stops after 500 more updates (`stop_at_step`), keeping a checkpoint at every $x + y$. Because the learning rate is constant, one 500-update branch covers all y . The measure stage evaluates the interpolation and ensemble losses for every cell.

```
experiments/a3_optimization/p1_mode_connectivity.py
BASE = TrainConfig(lr_schedule="wsd0.0") # constant after warmup

def branch(x, order_seed, base=BASE):
    fork = {}
    if x > 0:
        fork = dict(fork_from_run=training_run_name(prefix_for(base)),
                    fork_from_step=x)
    return replace(base, batch_order_seed=order_seed,
                  stop_at_step=x + max(BRANCH_STEPS),
                  keep_checkpoint_steps=tuple(x + y for y in BRANCH_STEPS),
                  **fork)
```

- (a) Set up the forked-run sweep. For each pair (x, y) , train one shared prefix for x steps, copy both the model weights and optimizer state, then continue two branches for y additional steps using the same constant learning rate, the same data distribution, and different data order. Sweep

$$x \in \{0, 50, 100, 250, 500, 1000\}, \quad y \in \{25, 50, 100, 250, 500\}.$$

For each cell, evaluate the two endpoint losses and the loss of a 0.5/0.5 prediction ensemble. For next-token probabilities p_{θ_1} and p_{θ_2} , use

$$p_{\text{ens}}(\cdot | c) = \frac{1}{2}p_{\theta_1}(\cdot | c) + \frac{1}{2}p_{\theta_2}(\cdot | c).$$

Plot endpoint loss and ensemble loss across the (x, y) grid.

- (b) On the same (x, y) grid, evaluate 0.5/0.5 weight averaging:

$$\theta_{\text{avg}} = \frac{1}{2}\theta_1 + \frac{1}{2}\theta_2.$$

For each cell, report the two endpoint losses, the weight-averaged loss $L(\theta_{\text{avg}})$, and the barrier $B(x, y)$. Plot $B(x, y)$ as a heatmap and mark the cells where $B(x, y) \leq 0.01$. From this heatmap, describe where the runs can and cannot be averaged.

- (c) Select representative cells from the (x, y) grid and evaluate the full interpolation curve

$$\theta(\alpha) = (1 - \alpha)\theta_1 + \alpha\theta_2, \quad \alpha \in \{0, 0.25, 0.5, 0.75, 1\}.$$

Plot validation loss versus α . Compare the interpolation curves to the midpoint barrier and to the 0.5/0.5 prediction ensemble.

- (d) Explore which training choices control whether models end up in the same low-loss basin, as assessed by the midpoint barrier and interpolation curves above. Try varying knobs such as learning rate, batch size, and learning-rate schedule. What patterns do you find, and which further experiments would help test your explanation?

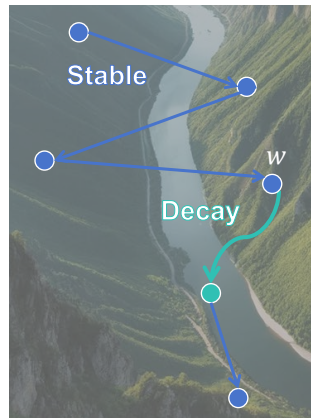
River-Valley Dynamics

Now let us turn our attention from minima to dynamics. How do the iterates behave during optimization in practice? We’ve seen some models of dynamics in the past (such as the noisy quadratic model, in our earlier assignment and lecture) that had some macroscopic predictive value for hyperparameters.

We want to now dig in a bit deeper, and look at fine-grained structure of the optimization landscape. One phenomenon we might be curious about is why learning rate schedules are so important to optimization in deep learning.

Even from the early days of deep learning, it was well known that schedulers such as the cosine schedule (Loshchilov and Hutter, 2017) would significantly improve optimization, even in cases like batch optimization, where we did not expect there to be a strong need for learning rate decay. Additionally, it was unclear how we should be reasoning about the optimal shape of learning rate decays, which theory told us should perhaps look like $O(1/\sqrt{t})$ or $O(1/t)$.

An emerging line of work (Wen et al., 2024; Dremov, Hägele, Kosson, and Jaggi, 2025) has been arguing that deep learning optimizers are often not predictably descending down a smooth landscape (with some noise, as NQM would predict) but instead have complicated, chaotic dynamics. Some components of θ_t may oscillate wildly back and forth, bouncing between “walls” in an optimization landscape, while other components may move more slowly and reliably over time. An evocative image of this process is the “river-valley” landscape, where the optimizer descends along a valley, but bouncing around on the sides of the valley.



The river-valley picture. With a large learning rate, iterates bounce between the steep hillsides while drifting along the river; decaying the learning rate brings them down to the river. Figure 1(a) of Wen, Li, Wang, Hall, Liang, and Ma, “Understanding Warmup-Stable-Decay Learning Rates: A River Valley Loss Landscape Perspective,” ICLR 2025.

We will investigate this phenomenon, first through the lens of learning rate schedules and smoothing, and then through the lens of the Hessian.

Problem 2: River-Valley Dynamics

Use the default depth-8, 614M-token setup, changing only the schedule and EMA settings. Compare validation losses at equal token counts. Define *regret* relative to a reference run as

$$R_M(t) = L(\theta_t^M) - L(\theta_t^{\text{ref}}).$$

The starter launches the four schedules for parts (a) and (b), with evaluation-only EMAs logged as

```

val_ema{decay}_loss:

experiments/a3_optimization/p2_river_valley.py
EMA_DECAYS = (0.99, 0.999)
SCHEDULE_RUNS = [
    TrainConfig(lr_schedule=lr_schedule, ema_decays=EMA_DECAYS)
    for lr_schedule in ("wsd0.0", "linear", "cos", "wsd0.2")
]

```

Here `wsd0.0` is constant after warmup and `wsd0.2` decays over the last 20% of training. For part (c), `lr_dip=(start, bottom, end, scale)` multiplies any schedule by a temporary dip, with times given as fractions of training. For part (d), `lr_schedule="invsqrt"` and `"sgdrT"` (with `min_lr_ratio`) implement the two schedules below, and `warmup_steps` fixes the warmup length in updates.

- Compare constant-after-warmup, linear, cosine, and plateau-then-decay schedules. Plot learning rate, training loss, and validation loss over training. Describe the loss shapes. To ensure your results are robust, you may want to tune your peak learning rate over alternatives, such as 1.5×10^{-3} and 6×10^{-3} . Use the run with the lowest final validation loss as the reference for part (b).
- Track an exponential moving average (EMA) of the model parameters during each run:

$$\bar{\theta}_0 = \theta_0, \quad \bar{\theta}_t = \beta_{\text{EMA}} \bar{\theta}_{t-1} + (1 - \beta_{\text{EMA}}) \theta_t.$$

Use EMA only for evaluation. Try a few averaging timescales across all schedules, and plot raw and EMA validation loss and regret. How much of the gap between schedules remains after averaging?

- Decay the learning rate partway through training, then warm it back to its previous level without resetting model or optimizer state. Vary the depth and duration of this intervention. Use a control without the intervention and give both runs the same final cooldown. Plot regret relative to this control. Does the loss improvement survive rewarmup, and what is the final regret? Is it possible to predict the duration and magnitude of regret based on hyperparameters such as the learning rate schedule?
- Test these two *horizon-free* learning-rate schedules, keeping AdamW unchanged:
 - Inverse-square-root decay** (Vaswani et al., 2017, Section 5.3): for update $t \geq 1$, use $\alpha_t = \alpha_{\max} \min\{t/w, \sqrt{w/t}\}$, where w is a fixed warmup length in updates.
 - Cosine warm restarts** (Loshchilov and Hutter, 2017, SGDR): after a fixed warmup, repeat $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})[1 + \cos(\pi s/T)]$, where s is the update index within a cycle of fixed length T . Restart only the learning rate, not the model or optimizer state.

Choose w , T , and learning-rate bounds independently of the final budget. Keep each schedule fixed across several stopping budgets, without adding a budget-specific cooldown, and compare validation loss and regret against schedules tuned to each budget. Are these useful alternatives when the training budget is unknown?

Hessian and Sharpness

We now study the river valley landscape structure in more detail. What structural factors in the loss landscape might be causing these behaviors? To understand this, we will take a bit of a detour in looking at a very basic convex optimization phenomenon, and its connection to deep learning training dynamics.

Consider gradient descent with step size η on a quadratic where the maximum eigenvalue of the hessian is $\lambda_{\max}$. If we have that $\eta\lambda_{\max} < 2$, the iterates will converge, while if we have $\eta\lambda_{\max} > 2$ then the iterates will oscillate and eventually blow up. The noisy quadratic model would suggest that $\eta < 2/\lambda_{\max}$ is necessary for convergence, even in a deep neural network.

However, [Cohen et al. \(2021\)](#) found that training does not work this way. In full-batch gradient descent on real networks, the sharpness $\lambda_{\max}$ tends to rise during training (a phenomenon known as *progressive sharpening*) until it reaches $2/\eta$, and then it stops rising and hovers there. This is often referred to as the “edge of stability” since the optimization process seems to find locations in the landscape where the dynamics are barely stable, and stay there.

In cases such as Adam, where we take a preconditioned gradient step, there is a similar quantity called the *preconditioned* sharpness ([Cohen et al., 2022](#)) which also progressively increases and stabilizes at a particular value dependent on β (for $\beta_1 = 0.9$, it is $38/\eta$).

These are clean results from small, full-batch experiments. In this problem you will check whether they hold for our transformer, and then see what is left of them in the minibatch training we actually do.

More formally, for parameters θ , the sharpness $\lambda_{\max}$ is the top eigenvalue of the Hessian H of the loss on a fixed probe set of sequences. For AdamW we have the preconditioned sharpness

$$\lambda_{\max}^{\text{pre}} = \lambda_{\max}\left(D^{-1/2}HD^{-1/2}\right), \quad D = \sqrt{\hat{\sigma}} + \epsilon,$$

where $\hat{\sigma}$ is AdamW’s bias-corrected second-moment estimate. For SGD and Adam, we expect $\eta\lambda_{\max}$ and $\eta\lambda_{\max}^{\text{pre}}$ respectively to sharpen and stabilize to a constant. In the classic edge-of-stability case, this stabilization point happens at the edge of stability for a quadratic, which happens at $\eta\lambda_{\max} < 2$ for SGD. For Adam with $\beta_1 = 0.9$ and a frozen preconditioner, this becomes $\eta\lambda_{\max}^{\text{pre}} < 2(1 + \beta_1)/(1 - \beta_1) = 38$. We will be computing all these quantities throughout the training process. Computing Hessians are quite computationally expensive, so we will have to scale down some of our experiments, and make use of clever computational tricks, such as using Lanczos iteration on Hessian-vector products to find the maximum eigenvalue. The starter code (below) has all of these available to use.

Problem 3: The Edge of Stability (22 runs)

This problem makes extensive use of Hessian eigenvalues, and helper code is available at `experiments/a3_optimization/p3_edge_of_stability.py`. It has two helpers that return a `TrainConfig` with the sharpness logger attached: `full_batch(N, updates, ...)` for part (a) and `minibatch(...)` for parts (b) and (c). `sharpness.py` is provided to read out Hessian statistics that are stored in weights and biases.

```
experiments/a3_optimization/p3_edge_of_stability.py
FULL_BATCH_RUNS = [
    full_batch(32, 1500, learning_rate=0.01), # (a) i: the clean case
]
MINIBATCH_RUNS = [
    minibatch(), # (b): the base recipe
]
```

Part (a) uses full-batch training on a few sequences, the setting of Cohen et al. Parts (b) and (c) use the minibatch training we actually do. Throughout, you will want to plot sharpness on a log axis, with a dashed line at the stability threshold.

- (a) **Replicating Cohen et al (Full Batch).** Use gradient descent on the first N training sequences with the 16-times embedding initialization, for 1,500 updates (`full_batch(N, 1500, learning_rate=...)`). (3 runs)
- The clean case.* Train on $N = 32$ at $\eta = 0.01$. Plot $\lambda_{\max}$ against the update, and in a second panel against the training loss. Describe what happens. Does this match the edge-of-stability predictions? Compare to plots in Cohen et al.
 - Changing η mid-run.* Rerun the clean case, but halve η at update 750 (`lr_schedule="step750x0.5"`). Then rerun it doubling η instead. Predict where $\lambda_{\max}$ goes in each case, then check. Is the response symmetric?
- (b) **Minibatch training.** Now study edge of stability in the minibatch case, similar to our standard d8 setting. `minibatch()` gives a simple recipe (but with constant LR and much shorter horizon, to save compute). Plot $\eta_t \lambda_{\max}^{\text{pre}}$ on the held-out probe against training tokens, leaving out warmup. Does minibatch AdamW train at the edge? (1 run)
- (c) **Open ended hyperparameter exploration.** Study the impact of hyperparameters on edge of stability. What factors affect a model's tendency to enter and stay at the edge of stability? (18 runs)

Inside the Hessian

In the edge-of-stability problem we saw that the top eigenvalue of the Hessian behaved in a very characteristic way in deep learning optimization. It turns out that the Hessian (both the top eigenvalue, as well as its eigenvalue spectrum) has been a key player in developing new optimization and learning algorithms, so we will go one step further to gain intuition about the Hessian.

Our objects of study

Directly constructing the Hessian is computationally infeasible. For a model with m parameters, the Hessian has m^2 entries. Therefore, we will focus our studies on a few key aspects of Hessians. For the eigenvalues, we are interested in the size of the maximal eigenvalue (sharpness), relative to the trace (normalized sharpness), negative eigenvalues (saddles) and the overall distribution.

Beyond eigenvalues, we are interested in the *block structure* of the Hessian. Off diagonal elements in the Hessian can be viewed as “dependencies” between variables (i.e. changing one variable requires that we change another to keep loss similar). To measure this, we will look at the submatrix of the Hessian H corresponding to parameter families, such as: input embedding, attention projections, QK-norm gains, MLP weights, other RMSNorm gains, and the output head. If the Frobenius norm of these Hessian submatrices is close to the overall Hessian one, this would mean that interactions between variables are mostly “within block”.

Tools

Hessians are computationally expensive, unruly objects to deal with, and so we will provide you with some standard tools to be able to perform this analysis. H is the Hessian of the mean next-token loss on a fixed *probe*, consisting of the 64 held-out training-source sequences used in part (b) of the edge-of-stability problem. To avoid numerical issues that can happen in our algorithms, we will use FP32 with TF32 off.

- **Lanczos** iterations on Hessian-vector products can compute top and bottom eigenpairs of H as well as the residual $\|Au - \lambda u\|/|\lambda|$. This was what we used to obtain the top eigenvalue in the edge-of-stability problem. We will make a more complete use of the information from Lanczos iteration, including its Ritz values to estimate the eigenvalue density.
- **Hutchinson**. We will often want to estimate the trace and squared trace of a Hessian matrix. This can be estimated easily using the Hutchinson estimator. For random sign vectors z , $\mathbb{E}[z^\top Az] = \text{tr } A$ and $\mathbb{E}[z \odot Az] = \text{diag } A$. Importantly, this can be used to estimate the fraction of the Hessian captured in a submatrix, since if z is a random sign vector on block b and zero elsewhere, $\mathbb{E}\|(Hz)_c\|^2 = \|H_{cb}\|_F^2$ for every block c .

Problem 4: Inside the Hessian (8–20 runs)

The starter is `experiments/a3_optimization/p4_inside_the_hessian.py`, with one stage per part (a, b-rescale, b-train, b-measure, c) and a fetch stage. The tools are in `hessian.py` in the same folder (`ProbeHessian(model, probe)` gives `H.hvp(v)`; `partition` and `restrict` give blocks), `hessian_measure.py` has `measure()` and part (c)’s `continue_projected()`, `hessian_checkpoints.py` loads the provided runs and your own, and `hessian_jobs.load_result(name)` reads a fetched result.

```
experiments/a3_optimization/p4_inside_the_hessian.py
LOOK_JOBS = [                                # part (a)
    {"name": f"a-adamw-{step}"},
```

```

    "measure": {"run": "adamw", "step": step, "extremes": True, "families": True,
               "energy": ("family", "layer"),
               "density_runs": 4 if step in DENSITY_AT else 0}}
    for step in CHECKPOINTS
]
SUBSPACE_JOBS = [
    # part (c)
    {"name": f"c-{arm}", "continue": {"**START", "transform": arm}}
    for arm in ("full", "top", "removed")
]

```

Checkpoints. The default d8 AdamW run and an SGD-with-momentum run (peak learning rate 1, momentum 0.9, no weight decay) are provided with optimizer state at 13 steps from 0 to 9,375 (614M tokens; step 61 is 4M tokens, 153 is 10M, 305 is 20M, 1,526 is 100M).

A Hessian-vector product on the probe takes about 2 seconds on an H100, so plan measurements by counting products: a part (a) checkpoint takes about 13 minutes with these options (a density adds 13 more), and part (c) about 20 minutes per 50 steps. Always use all 64 probe rows.

(a) **What the Hessian looks like** (0 runs). Using the AdamW checkpoints, describe how the Hessian changes over training.

- *Which family is sharpest? How does that change?* Plot each family block's top eigenvalue, $\lambda_{\max}$ and $\text{tr}(H^2)/d$ against tokens.
- *What does the spectrum look like?* Estimate the eigenvalue density at initialization, 10M, 100M and 614M tokens. Are there outliers? Negative eigenvalues? What are the implications for training?
- *How block-diagonal is it?* Split $\text{tr}(H^2) = \|H\|_F^2$ into the parts inside and between blocks, by family and by layer, over training. Compare with a random partition of the parameters into blocks of the same sizes. Does curvature grow with depth?

(b) **Move $\lambda_{\max}$ without moving the loss** (about 4–10 runs). Across three seeds the default recipe ends with $\lambda_{\max}$ between 4.5 and 6.0 and a validation loss between 2.925 and 2.931. Find changes to the training configuration that change the final $\lambda_{\max}$ by at least $2\times$ while keeping the final validation loss inside that range. Explain your rationale for the interventions you make.

- *Starting point.* Is there a way to change some of the weights so that the model makes the same predictions while its Hessian changes? (Hint: think back to our lecture discussion on loss invariance). Find such a transformation in our architecture, apply it by hand to the final checkpoint, and measure the loss and the curvature of the affected blocks before and after.
- *Sharpening in training.* Find a change to the training recipe that will induce this sharpening effect without manual intervention. Hyperparameter interventions alone may get you a decent way, but the strongest interventions may involve some code changes as well. What does this mean if we want to compare $\lambda_{\max}$ across runs?

(c) **Training in and out of the top subspace** (about 3–10 short runs). [Gur-Ari et al. \(2018\)](#) found that the gradient lies mostly in the span of the top few Hessian eigenvectors. Test to see if this holds in our d8 system using SGD with momentum. (We switch from Adam to SGD, as it's much easier to reason about the relationship between the Hessian and our updates).

- *Starting point.* From the SGDM checkpoint at 100M tokens, zero the momentum and continue 300 steps with three updates: the full update, the update projected onto the top-10 eigenvectors, and the update with that component removed. Recompute the eigenvectors

every 25 steps. Log the losses, the fraction of the gradient in the subspace, and $\lambda_{\max}$ against the heavy-ball threshold $2(1 + \beta)/\eta_t$.

- *Investigate the phenomenon.* Some of these results may be unexpected. Vary a range of hyperparameters of your choosing (e.g. k , refresh interval, learning rate, starting point) and see if your conclusions change. Do any changes make the training faster? What is your mental model of how eigendirections in the Hessian relate to the optimizer trajectory?

Optional exploration

To explore the topics in this assignment more broadly, construct your own prediction problem. This final exercise is not related to a quiz question, but can lead to bonus points.

Problem 5: Your own prediction problem (5-10 runs)

You have now explored the phenomena in this assignment thoroughly, and may have found unexpected or surprising phenomena. We would like to help make better, more interesting quizzes based on your insights.

Include the following in your exploration:

- (a) A question writeup, similar to the example questions or earlier quizzes. Answers should be objectively checkable based on the execution of a piece of code (ideally, multiple choice or numerical prediction).
- (b) A code diff and executable code which resolves to an answer in the question (e.g. prints the loss). The code diff should be compact (<100 lines of code)

Aim for a fair, novel question: its outcome should be possible to reason about using the assignment, while testing an insight beyond the supplied examples. Course staff may use interesting questions in a future quiz.